

Contents

Estudio del proyecto Philosophers — Trazado completo	2
Tabla de contenidos	2
1. El problema: cena de los filósofos	2
2. Visión general de la arquitectura	3
Hilos	3
Recursos compartidos	3
Diagrama de flujo (alto nivel)	3
3. Estructuras de datos (philo.h)	4
t_fork	4
t_philo	4
t_data	4
4. Trazado completo desde main	5
main (main.c)	5
5. Parseo y validación de argumentos	6
parse_args (parse.c)	6
validate_args (parse.c)	6
is_all_digits (parse.c)	7
ft_atol (parse.c)	8
6. Inicialización (init_data)	8
init_data (init.c)	8
init_forks (init.c)	9
init_philos (init.c)	10
init_mutexes (init.c)	11
7. Arranque de la simulación (start_simulation)	11
start_simulation (init.c)	11
create_philo_threads (init.c)	13
8. La rutina del filósofo (philo_routine)	14
philo_routine (routine.c)	14
is_over (routine.c)	15
threads_ready (routine.c)	15
wait_for_start (routine.c)	16
lone_philo (routine.c)	16
stagger_start (routine.c)	16
9. El ciclo eat → sleep → think	17
philo_eat (actions.c)	17
take_forks (actions.c)	18
put_forks (actions.c)	19
philo_sleep (actions.c)	19
philo_think (actions.c)	19
compute_think_time (actions.c)	20
10. El monitor (monitor_routine)	21
monitor_routine (monitor.c)	21
check_death (monitor.c)	21
announce_death (monitor.c)	22
check_all_ate (monitor.c)	23
11. Utilidades (tiempo, impresión, errores)	23
get_time_ms (utils.c)	23
get_elapsed (utils.c)	24
ft_usleep (utils.c)	24
print_status (utils.c)	25
error_exit (utils.c)	26
ft_strlen_local (utils.c)	26
12. Limpieza y salida	26
cleanup (init.c)	26

13. Decisiones de diseño y por qué	27
Lock-ordering global por id	27
Barrera de arranque	27
print con write+snprintf	28
Polling fino del monitor (200µs)	28
Think time dinámico	28
14. Preguntas frecuentes en defensa	28
¿Cómo evitas el deadlock?	28
¿Cómo evitas el starvation?	28
¿Cómo detectas la muerte en <10ms?	28
¿Cómo evitas que un filo imprima un mensaje después de "died"?	28
¿Hay data races?	28
¿Hay riesgo de deadlock entre los mutexes globales?	29
¿Qué hace last_meal_time = start_time al inicio?	29
¿Por qué > y no >= en check_death?	29
¿Por qué philo_eat actualiza last_meal_time al PRINCIPIO?	29
¿Funciona con 1 filósofo?	29
¿Funciona con N = 200 (máximo)?	29
¿Por qué no usar sched_yield o variables de condición?	29
¿Hay memory leaks?	30
¿Cumple la Norma de 42?	30
Apéndice: ejemplo de output esperado	30

Estudio del proyecto Philosophers — Trazado completo

Documento de estudio para defender el proyecto. Pensado para leer de principio a fin: traza cada función desde que main arranca hasta que el programa retorna, entrando recursivamente en cada llamada. Cada función incluye: qué hace, por qué hace eso, en qué orden, y los detalles finos que un evaluador puede preguntar.

Tabla de contenidos

1. El problema: cena de los filósofos
2. Visión general de la arquitectura
3. Estructuras de datos (philo.h)
4. Trazado completo desde main
5. Parseo y validación de argumentos
6. Inicialización (init_data)
7. Arranque de la simulación (start_simulation)
8. La rutina del filósofo (philo_routine)
9. El ciclo eat → sleep → think
10. El monitor (monitor_routine)
11. Utilidades (tiempo, impresión, errores)
12. Limpieza y salida
13. Decisiones de diseño y por qué
14. Preguntas frecuentes en defensa

1. El problema: cena de los filósofos

Hay N filósofos sentados en una mesa redonda. Entre cada par de filósofos hay UN tenedor (no dos). Para comer, un filósofo necesita los DOS tenedores adyacentes — el de su izquierda y el de su derecha. Los filósofos se turnan entre tres estados:

- **Eating:** están comiendo (mantienen los dos tenedores)
- **Sleeping:** están durmiendo (sin tenedores)
- **Thinking:** están pensando (sin tenedores, en cualquier momento pueden intentar coger los tenedores otra vez)

Reglas del subject:

- Si un filósofo no empieza una comida en `time_to_die` ms desde su última comida (o desde el inicio si todavía no ha comido), MUERE.
- La simulación termina cuando: (a) muere un filósofo, o (b) si se proporciona el parámetro `must_eat`, todos han comido al menos esa cantidad de veces.
- El mensaje de muerte debe imprimirse en menos de 10 ms desde que ocurre.
- Cada filósofo es un hilo. Los tenedores son mutex.

Argumentos del programa:

`./philo <nb_philos> <time_to_die> <time_to_eat> <time_to_sleep> [must_eat]`

Ejemplo: `./philo 5 800 200 200 7` → 5 filósofos, 800 ms para morir, comer dura 200 ms, dormir dura 200 ms, cada filósofo debe comer 7 veces.

Los dos problemas clásicos que hay que evitar:

1. **Deadlock:** todos cogen su tenedor izquierdo, nadie puede coger el derecho porque su vecino tiene el suyo izquierdo (que es el derecho del que está al lado). El sistema se queda congelado.
2. **Starvation:** un filósofo concreto nunca consigue los dos tenedores porque sus vecinos siempre lo hacen antes. Acaba muriendo.

2. Visión general de la arquitectura

Hilos

- **N hilos de filósofos:** cada uno ejecuta `philo_routine`. Hace el ciclo `eat` → `sleep` → `think` indefinidamente hasta que la simulación termina.
- **1 hilo monitor:** ejecuta `monitor_routine`. Vigila quién ha comido hace cuánto, y detecta muertes o el cumplimiento del `must_eat`.

Recursos compartidos

- **N tenedores:** cada uno es un `pthread_mutex_t`. Sólo un filósofo puede tener un tenedor a la vez.
- **3 mutexes globales:**
 - `print_mutex`: protege `printf/write` a `stdout`. Sin él, los mensajes se mezclarían.
 - `stop_mutex`: protege la flag `stop` (¿ha terminado la simulación?) y `threads_ready` (¿está `main` listo para que arranquen los hilos?).
 - `meal_mutex`: protege `last_meal_time` y `meals_eaten` de cada filósofo. Los hilos lo escriben, el monitor lo lee.

Diagrama de flujo (alto nivel)

```
main
├── parse_args      (parsea + valida argv)
├── init_data       (malloc + init mutexes + asigna tenedores)
├── start_simulation
│   ├── crea N hilos de filósofos (cada uno espera en barrera)
│   ├── set start_time + last_meal_time bajo meal_mutex
│   ├── crea hilo monitor
│   └── set threads_ready = 1 (libera barrera)
```

```

┌───┬─── pthread_join monitor (bloquea hasta muerte o all_ate)
└───┬─── pthread_join N filósofos
    └─── cleanup (destroy mutexes + free)

```

Cada hilo de filósofo:

```

philo_routine
├── wait_for_start (barrera)
├── lone_philo si N==1
├── stagger_start (desfase inicial)
└── bucle: philo_eat → philo_sleep → philo_think

```

El monitor:

```

monitor_routine
├── wait threads_ready
└── bucle (cada 200µs): check_death | check_all_ate

```

3. Estructuras de datos (philo.h)

t_fork

```

typedef struct s_fork
{
    pthread_mutex_t mutex;
    int id;
} t_fork;

```

Un tenedor = un mutex + un id. El id sirve para el **lock-ordering**: cuando un filósofo necesita coger 2 tenedores, los coge SIEMPRE en orden creciente de id. Esto evita el deadlock (lo explicaré en init_philos).

t_philo

```

typedef struct s_philo
{
    int id; // 1..N
    int meals_eaten; // cuántas veces ha comido
    long long last_meal_time; // timestamp ms de su última comida
    pthread_t thread;
    t_fork *first_fork; // tenedor a coger primero (menor id)
    t_fork *second_fork; // tenedor a coger segundo (mayor id)
    t_data *data; // puntero al contexto global
} t_philo;

```

first_fork y second_fork no son izquierdo/derecho — son el de **menor id** y el de **mayor id**. Esto es clave para el lock-ordering.

t_data

```

struct s_data
{
    int nb_philos;
    long long time_to_die;
    long long time_to_eat;
}

```

```

long long    time_to_sleep;
int          must_eat;        // -1 si no se especifica
long long    start_time;      // timestamp ms del arranque global
int          stop;            // ¿ha terminado la simulación?
int          threads_ready;   // ¿pueden arrancar los hilos?
pthread_mutex_t print_mutex;
pthread_mutex_t stop_mutex;
pthread_mutex_t meal_mutex;
t_fork       *forks;          // array de N tenedores
t_philo       *philos;        // array de N filósofos
};

```

Por qué must_eat es int y -1 significa "no aplica": el subject permite no pasar este parámetro. Usamos -1 como centinela porque ningún valor válido puede ser negativo (lo validamos en parse).

Por qué los tiempos son long long: los timestamps en ms desde epoch no caben en int. gettimeofday devuelve segundos como time_t y microsegundos como suseconds_t; al multiplicar segundos × 1000 necesitamos al menos 64 bits.

4. Trazado completo desde main

main (main.c)

```

int main(int argc, char **argv)
{
    t_data data;

    data = (t_data){0};
    if (parse_args(&data, argc, argv) != 0)
        return (1);
    if (init_data(&data) != 0)
        return (1);
    if (start_simulation(&data) != 0)
    {
        cleanup(&data);
        return (1);
    }
    cleanup(&data);
    return (0);
}

```

Qué hace, línea por línea:

1. `data = (t_data){0};` — inicializa toda la estructura a ceros. Esto es importante porque `cleanup()` puede llamarse en cualquier punto y chequea si `data.forks` o `data.philos` son NULL antes de liberar. Si no inicializáramos a cero, tendrían basura y `cleanup` rompería.
2. `parse_args(&data, argc, argv)` — parsea y valida `argv`. Si falla, imprime el error y devuelve 1. Aquí salimos con `return 1` SIN llamar a `cleanup` porque no se ha hecho ningún `malloc` ni `mutex_init` todavía.
3. `init_data(&data)` — hace los `malloc` y los `pthread_mutex_init`. Si falla, internamente llama a `cleanup` para liberar lo que sí se inicializó y devuelve 1.
4. `start_simulation(&data)` — arranca los hilos y bloquea hasta que la simulación termine. Si `pthread_create` falla aquí dentro, libera lo que pueda y devuelve 1. Por eso necesitamos `cleanup` también en la rama de error de `start_simulation`.

5. `cleanup(&data)` — destruye mutexes y libera memoria.

6. `return 0;` — todo correcto.

Preguntas posibles en defensa:

- ¿Por qué `data = (t_data){0}` y no `memset`? Es un compound literal inicializado a cero — C99 estándar, equivalente a `memset` pero sin depender de `<string.h>` ni llamar a una función.
- ¿Por qué no usar `static t_data data`? Variables `static` son globales en memoria. La Norma de 42 no las permite y además queremos que `data` viva en el stack de `main`, no en la zona BSS.

5. Parseo y validación de argumentos

`parse_args (parse.c)`

```
int parse_args(t_data *data, int argc, char **argv)
{
    if (validate_args(argc, argv) != 0)
        return (1);
    data->nb_philos = (int)ft_atol(argv[1]);
    data->time_to_die = ft_atol(argv[2]);
    data->time_to_eat = ft_atol(argv[3]);
    data->time_to_sleep = ft_atol(argv[4]);
    data->must_eat = -1;
    if (argc == 6)
        data->must_eat = (int)ft_atol(argv[5]);
    return (0);
}
```

Flujo:

1. Llama a `validate_args` que hace toda la validación. Si falla, ya se ha impreso el error y nos limitamos a devolver 1.
2. Si pasa la validación, ya sabemos que todos los argumentos son enteros positivos válidos, así que podemos hacer `ft_atol` con seguridad.
3. `must_eat = -1` por defecto (centinela "no aplica"). Si hay 6 args (incluyendo `argv[0]`), parseamos el quinto.

`validate_args (parse.c)`

```
int validate_args(int argc, char **argv)
{
    long long val;
    int i;

    if (argc < 5 || argc > 6)
        return (error_exit("Usage: ./philo n ttd tte tts [n_eat]"));
    i = 1;
    while (i < argc)
    {
        if (!is_all_digits(argv[i]))
            return (error_exit("Error: arguments must be positive integers"));
        i++;
    }
    val = ft_atol(argv[1]);
    if (val < 1 || val > 200)
```

```

        return (error_exit("Error: number of philosophers must be 1-200"));
    i = 2;
    while (i <= 4)
    {
        val = ft_atol(argv[i]);
        if (val < 1 || val == LLONG_MAX)
            return (error_exit("Error: time values must be >= 1"));
        i++;
    }
    if (argc == 6)
    {
        val = ft_atol(argv[5]);
        if (val < 1 || val == LLONG_MAX)
            return (error_exit("Error: number of meals must be >= 1"));
    }
    return (0);
}

```

Validaciones en orden:

1. **Cantidad de args:** argc < 5 (faltan args) o argc > 6 (sobran). Argc mínimo = 5 porque [0]=programa, [1..4]=los 4 obligatorios. Argc máximo = 6 con el must_eat opcional.
2. **Cada arg solo dígitos:** is_all_digits rechaza signos, espacios, letras. Aceptar +5 o -5 o 5 haría el parser más complejo y el subject pide enteros positivos.
3. **Rango de filósofos:** 1..200. El subject permite hasta 200. Más es irrelevante para el eval y daría problemas de scheduler.
4. **Tiempos ≥ 1:** si time_to_die = 0, todos mueren instantáneamente. Si time_to_eat = 0, eat es instantáneo y el subject no lo contempla. Igual para sleep.
5. **must_eat ≥ 1** si se proporciona: comer 0 veces no tiene sentido.
6. **LLONG_MAX detección:** ft_atol devuelve LLONG_MAX cuando hay overflow. Si el usuario pasa 99999999999999999999 lo detectamos.

is_all_digits (parse.c)

```

static int is_all_digits(const char *s)
{
    if (!s || !*s)
        return (0);
    while (*s)
    {
        if (*s < '0' || *s > '9')
            return (0);
        s++;
    }
    return (1);
}

```

- Rechaza NULL y string vacío.
- Acepta solo caracteres '0'..'9'. Ni signos, ni espacios, ni nada.
- Es static porque solo se usa en este archivo.

ft_atol (parse.c)

```
long long ft_atol(const char *str)
{
    long long    result;
    long long    digit;

    result = 0;
    if (!str || !*str)
        return (-1);
    while (*str >= '0' && *str <= '9')
    {
        digit = *str - '0';
        if (result > (LLONG_MAX - digit) / 10)
            return (LLONG_MAX);
        result = result * 10 + digit;
        str++;
    }
    if (*str != '\0')
        return (-1);
    return (result);
}
```

Detección de overflow: la clave está en `result > (LLONG_MAX - digit) / 10`. Reescribamos: queremos saber si `result * 10 + digit > LLONG_MAX`. Despejando: `result > (LLONG_MAX - digit) / 10`. Lo comprobamos ANTES de hacer la multiplicación para no overflowear el chequeo mismo.

Diferencia con la atol estándar:

- atol estándar acepta signos (-5, +5). La nuestra solo dígitos puros porque la pre-validación con `is_all_digits` ya lo garantiza, pero por seguridad mantenemos el comportamiento estricto.
- atol estándar tiene comportamiento indefinido en overflow. La nuestra devuelve `LLONG_MAX`, que es un valor centinela detectable.

6. Inicialización (init_data)

init_data (init.c)

```
int init_data(t_data *data)
{
    data->stop = 0;
    data->threads_ready = 0;
    data->start_time = 0;
    data->forks = malloc(data->nb_philos * sizeof(t_fork));
    if (!data->forks)
        return (error_exit("malloc forks failed"));
    data->philos = malloc(data->nb_philos * sizeof(t_philo));
    if (!data->philos)
    {
        free(data->forks);
        data->forks = NULL;
        return (error_exit("malloc philos failed"));
    }
    if (init_forks(data) != 0)
```

```

    return (cleanup(data), 1);
if (init_philos(data) != 0)
    return (cleanup(data), 1);
if (init_mutexes(data) != 0)
    return (cleanup(data), 1);
return (0);
}

```

Orden de inicialización (importante para cleanup):

1. Setea flags a cero (stop, threads_ready, start_time).
2. malloc del array de tenedores.
3. malloc del array de filósofos. Si falla → libera forks y sale.
4. init_forks → llama a pthread_mutex_init para CADA tenedor.
5. init_philos → asigna ids, tenedores y rellena los structs de filos.
6. init_mutexes → init de los 3 mutexes globales (print, stop, meal).

Por qué este orden: los tenedores deben estar inicializados ANTES de que init_philos los referencie (técnicamente solo guarda punteros, no los usa, pero es buena práctica). Los mutexes globales se inicializan al final porque los hilos no arrancan hasta start_simulation, así que el orden de los globales no importa demasiado, pero los ponemos al final para que el cleanup en caso de fallo sea más simple.

El truco return (cleanup(data), 1); es la coma de C: ejecuta cleanup(data) y luego retorna 1. Es legal y compacto.

init_forks (init.c)

```

static int init_forks(t_data *data)
{
    int i;

    i = 0;
    while (i < data->nb_philos)
    {
        data->forks[i].id = i;
        if (pthread_mutex_init(&data->forks[i].mutex, NULL) != 0)
        {
            while (i > 0)
                pthread_mutex_destroy(&data->forks[--i].mutex);
            return (error_exit("pthread_mutex_init fork failed"));
        }
        i++;
    }
    return (0);
}

```

Detalle del rollback: si pthread_mutex_init falla en el tenedor i, los tenedores 0..i-1 ya están inicializados y hay que destruirlos. El bucle while (i > 0) pthread_mutex_destroy(&data->forks[--i].mutex) hace pre-decremento de i: primero decrementa, luego accede. Así pasa por i-1, i-2, ... 0 sin tocar i (que falló).

Por qué id = i (0-indexado): el id del tenedor es su índice en el array. Esto facilita el lock-ordering posterior.

init_philos (init.c)

```
static int init_philos(t_data *data)
{
    int i;
    t_fork *left;
    t_fork *right;

    i = 0;
    while (i < data->nb_philos)
    {
        data->philos[i].id = i + 1;
        data->philos[i].meals_eaten = 0;
        data->philos[i].last_meal_time = 0;
        data->philos[i].data = data;
        left = &data->forks[i];
        right = &data->forks[(i + 1) % data->nb_philos];
        if (left->id < right->id)
        {
            data->philos[i].first_fork = left;
            data->philos[i].second_fork = right;
        }
        else
        {
            data->philos[i].first_fork = right;
            data->philos[i].second_fork = left;
        }
        i++;
    }
    return (0);
}
```

Asignación de tenedores:

- El filósofo i (en 0-indexado interno) tiene como vecino:
 - Tenedor izquierdo = `forks[i]`
 - Tenedor derecho = `forks[(i+1) % N]`

El módulo es para cerrar el círculo: el último filósofo tiene como derecho el `forks[0]`.

Por qué el filósofo en sí está 1-indexado (`id = i + 1`): porque el subject pide imprimir los mensajes con `id 1..N`. Internamente trabajamos con `0..N-1`, pero al imprimir sumamos 1.

El núcleo: lock-ordering

```
if (left->id < right->id)
    first_fork = left, second_fork = right;
else
    first_fork = right, second_fork = left;
```

Para CADA filósofo, `first_fork` es el de MENOR id. Esto es el truco crítico que rompe el deadlock.

Ejemplo con 5 filósofos:

Filo (id)	Left fork	Right fork	First (lock 1º)	Second
1	fork[0]	fork[1]	fork[0]	fork[1]
2	fork[1]	fork[2]	fork[1]	fork[2]
3	fork[2]	fork[3]	fork[2]	fork[3]
4	fork[3]	fork[4]	fork[3]	fork[4]

Filo (id)	Left fork	Right fork	First (lock 1º)	Second
5	fork[4]	fork[0]	fork[0]	fork[4]

Fíjate en el filo 5: su izquierdo es fork[4] y su derecho es fork[0]. Como fork[0] < fork[4], el filo 5 coge primero el fork[0] (su derecho), no el izquierdo. Los demás siempre cogen primero el izquierdo. El filo 5 ROMPE la simetría circular.

Por qué esto previene deadlock:

El deadlock clásico ocurre cuando todos cogen su izquierdo a la vez, quedándose esperando el derecho que su vecino derecho tiene. Aquí no puede pasar porque el filo N (último) coge primero fork[0], pero el filo 1 también necesita fork[0] como first_fork. Solo uno de los dos lo consigue, el otro espera. No hay forma de que todos cojan un tenedor a la vez.

Más formalmente: el lock-ordering global por id elimina ciclos en el grafo de wait-for. Si todos los hilos cogen recursos en el mismo orden (menor id antes que mayor id), no puede formarse un ciclo de espera.

init_mutexes (init.c)

```
static int init_mutexes(t_data *data)
{
    if (pthread_mutex_init(&data->print_mutex, NULL) != 0)
        return (error_exit("pthread_mutex_init print failed"));
    if (pthread_mutex_init(&data->stop_mutex, NULL) != 0)
    {
        pthread_mutex_destroy(&data->print_mutex);
        return (error_exit("pthread_mutex_init stop failed"));
    }
    if (pthread_mutex_init(&data->meal_mutex, NULL) != 0)
    {
        pthread_mutex_destroy(&data->print_mutex);
        pthread_mutex_destroy(&data->stop_mutex);
        return (error_exit("pthread_mutex_init meal failed"));
    }
    return (0);
}
```

Inicialización secuencial con rollback explícito en caso de fallo. Pattern clásico: si X falla, destruir lo anterior.

Los 3 mutexes:

- **print_mutex**: protege stdout. Sin él, varios hilos podrían imprimir mensajes simultáneamente y se mezclarían.
- **stop_mutex**: protege stop y threads_ready. Sin él podríamos tener una data race entre el monitor leyendo stop y un filo escribiéndolo.
- **meal_mutex**: protege last_meal_time y meals_eaten de los filósofos. Los hilos escriben, el monitor lee. Sin lock, el monitor podría leer un valor parcialmente actualizado.

7. Arranque de la simulación (start_simulation)

start_simulation (init.c)

```

int start_simulation(t_data *data)
{
    pthread_t    monitor;
    int          i;

    if (create_philo_threads(data) != 0)
        return (1);
    pthread_mutex_lock(&data->meal_mutex);
    data->start_time = get_time_ms();
    i = 0;
    while (i < data->nb_philos)
    {
        data->philos[i].last_meal_time = data->start_time;
        i++;
    }
    pthread_mutex_unlock(&data->meal_mutex);
    if (pthread_create(&monitor, NULL, monitor_routine, data) != 0)
    {
        pthread_mutex_lock(&data->stop_mutex);
        data->stop = 1;
        data->threads_ready = 1;
        pthread_mutex_unlock(&data->stop_mutex);
        i = 0;
        while (i < data->nb_philos)
            pthread_join(data->philos[i++].thread, NULL);
        return (error_exit("pthread_create monitor failed"));
    }
    pthread_mutex_lock(&data->stop_mutex);
    data->threads_ready = 1;
    pthread_mutex_unlock(&data->stop_mutex);
    pthread_join(monitor, NULL);
    i = 0;
    while (i < data->nb_philos)
        pthread_join(data->philos[i++].thread, NULL);
    return (0);
}

```

Esta es probablemente la función más importante del proyecto. Vamos paso a paso.

Paso 1: Crear los hilos de filósofos (create_philo_threads)

Cada hilo arranca en philo_routine pero lo primero que hace es esperar en una barrera (wait_for_start) hasta que threads_ready == 1. Así que después de create_philo_threads, los N hilos están vivos pero bloqueados, esperando.

Paso 2: Setear start_time y last_meal_time bajo meal_mutex

```

pthread_mutex_lock(&data->meal_mutex);
data->start_time = get_time_ms();
for each philo: philos[i].last_meal_time = start_time;
pthread_mutex_unlock(&data->meal_mutex);

```

Esto es atómico: tomamos meal_mutex, registramos start_time, y ponemos last_meal_time = start_time para todos los filósofos. Soltamos el lock.

Por qué bajo meal_mutex y no sin lock: porque el monitor también lee last_meal_time bajo meal_mutex. Aunque el monitor no puede ejecutar todavía (está esperando a threads_ready), técnicamente lo escribiremos en una zona protegida por el mismo lock que su lector, lo cual es correcto

desde el principio.

Por qué `start_time = now` y no `start_time = 0`: porque `get_elapsed` devuelve `now - start_time`. Si `start_time` fuera 0, el monitor vería `elapsed = now` que es un número enorme (ms desde epoch) y mataría a todos los hilos instantáneamente.

Paso 3: Crear el monitor

```
pthread_create(&monitor, NULL, monitor_routine, data);
```

El monitor también espera en una barrera al principio.

Paso 4: Liberar la barrera

```
pthread_mutex_lock(&data->stop_mutex);  
data->threads_ready = 1;  
pthread_mutex_unlock(&data->stop_mutex);
```

Esto despierta a todos los hilos (hilos + monitor) a la vez. **Esta es la sincronización CLAVE:** todos los hilos arrancan con el mismo `start_time` y con `last_meal_time` ya inicializado.

Sin esta barrera, el primer hilo creado podría comer durante 200ms mientras los demás se están todavía creando. Cuando el monitor arranca, ve que el hilo N todavía tiene `last_meal_time = 0` (basura) o un valor muy antiguo, y lo mata por error.

Paso 5: Bloquear hasta que la simulación termine

```
pthread_join(monitor, NULL);
```

Main se queda dormido aquí. El monitor es quien decide cuándo termina todo: pone `stop = 1` y retorna cuando detecta una muerte o un all-ate.

Paso 6: Unir los hilos de filósofos

```
for each philo: pthread_join(philos[i].thread, NULL);
```

Una vez el monitor retorna, ya tenemos `stop = 1`. Los hilos lo detectarán en su próxima llamada a `is_over()` y saldrán de sus bucles. Los unimos uno a uno.

Manejo de error en `pthread_create` del monitor:

Si `pthread_create(monitor)` falla, los N hilos ya están corriendo esperando en la barrera. Tenemos que: (1) poner `stop=1` y `threads_ready=1` para que salgan, (2) hacer join de cada uno, (3) devolver error.

`create_philo_threads (init.c)`

```
static int create_philo_threads(t_data *data)  
{  
    int i;  
  
    i = 0;  
    while (i < data->nb_philos)  
    {  
        if (pthread_create(&data->philos[i].thread, NULL,  
                           philo_routine, &data->philos[i]) != 0)  
        {  
            pthread_mutex_lock(&data->stop_mutex);  
            data->stop = 1;  
            data->threads_ready = 1;  
            pthread_mutex_unlock(&data->stop_mutex);  
            while (--i >= 0)
```

```

        pthread_join(data->philos[i].thread, NULL);
        return (error_exit("pthread_create philo failed"));
    }
    i++;
}
return (0);
}

```

- Crea los N hilos uno a uno.
- Cada hilo se queda en wait_for_start hasta que main libere la barrera (o ponga stop=1).
- Si pthread_create falla en el i-ésimo, hay que liberar a los 0..i-1 que ya están vivos. Setea stop=1 y threads_ready=1 para que salgan de la barrera, y luego hace join.

8. La rutina del filósofo (philo_routine)

philo_routine (routine.c)

```

void *philo_routine(void *arg)
{
    t_philo *philo;
    t_data *data;

    philo = (t_philo *)arg;
    data = philo->data;
    wait_for_start(data);
    if (data->nb_philos == 1)
    {
        lone_philo(philo);
        return (NULL);
    }
    stagger_start(philo);
    while (!is_over(data))
    {
        if (philo_eat(philo))
            break ;
        if (philo_sleep(philo))
            break ;
        if (philo_think(philo, 0))
            break ;
    }
    return (NULL);
}

```

Casting void * → t_philo *: la signatura de pthread_create exige void (*)(void *). Le pasamos &data->philos[i] así que aquí casteamos de vuelta.

Paso 1: Esperar en la barrera

```
wait_for_start(data);
```

El hilo se queda aquí hasta que main marca threads_ready = 1. Es una espera activa con usleep(50) entre chequeos — barata.

Paso 2: Caso especial de 1 filósofo

```
if (data->nb_philos == 1) { lone_philo(philos); return NULL; }
```

Solo hay un filósofo. Como necesita 2 tenedores y solo hay 1, no puede comer nunca. Tiene que coger 1 tenedor, esperar y morir.

Paso 3: Stagger inicial

```
stagger_start(philos);
```

Algunos filos duermen un poco al principio para evitar que todos intenten coger tenedores a la vez. Explicado en detalle abajo.

Paso 4: Bucle principal

```
while (!is_over(data))  
{  
    if (philos_eat(philos)) break;  
    if (philos_sleep(philos)) break;  
    if (philos_think(philos, 0)) break;  
}
```

El ciclo de vida del filósofo: comer → dormir → pensar → repetir.

Cada función devuelve 1 si la simulación terminó (para salir del bucle) o 0 para continuar. El `if(...)` `break;` es para salir LO ANTES POSIBLE en cuanto detectemos `is_over == true`.

is_over (routine.c)

```
int is_over(t_data *data)  
{  
    int over;  
  
    pthread_mutex_lock(&data->stop_mutex);  
    over = data->stop;  
    pthread_mutex_unlock(&data->stop_mutex);  
    return (over);  
}
```

Lee la flag stop bajo stop_mutex. Devuelve su valor.

Por qué con lock: sin lock sería una data race (varios hilos lectores y un hilo escritor concurrentes). En la práctica un int se lee y escribe atómicamente en x86, pero según el estándar C eso es UB. El lock hace el código portable y correcto formalmente.

threads_ready (routine.c)

```
int threads_ready(t_data *data)  
{  
    int ready;  
  
    pthread_mutex_lock(&data->stop_mutex);  
    ready = data->threads_ready || data->stop;  
    pthread_mutex_unlock(&data->stop_mutex);  
    return (ready);  
}
```

Devuelve 1 si o bien threads_ready es 1 (caso normal: arranque liberado) o si stop es 1 (caso de error: hay que salir de la barrera sin haber arrancado de verdad).

Por qué incluye `data->stop`: si `pthread_create` falla y `main` quiere abortar, pone `stop = 1`. Sin la cláusula `stop`, los hilos que están en la barrera quedarían bloqueados para siempre.

wait_for_start (routine.c)

```
static void wait_for_start(t_data *data)
{
    while (!threads_ready(data))
        usleep(50);
}
```

Spin-wait con backoff de 50µs. Muy barato. Cuando `main` libera la barrera, en menos de 50µs todos los hilos lo detectan y arrancan.

lone_philo (routine.c)

```
static void lone_philo(t_philo *philo)
{
    t_data *data;

    data = philo->data;
    pthread_mutex_lock(&philo->first_fork->mutex);
    print_status(philo, "has taken a fork");
    ft_usleep(data->time_to_die + 10, data);
    pthread_mutex_unlock(&philo->first_fork->mutex);
}
```

Caso especial obligatorio del subject. Con 1 filósofo solo hay 1 tenedor. El filo lo coge, imprime "has taken a fork", duerme `time_to_die + 10` ms, y suelta.

Por qué `time_to_die + 10`: queremos asegurar que el monitor tenga tiempo de detectar la muerte y reportarla. El filo no debería terminar ANTES de que el monitor reporte. Si `ft_usleep` se interrumpe por `is_over`, sale antes. Si no se interrumpe, espera lo suficiente para que el monitor reporte la muerte y ponga `stop=1`.

stagger_start (routine.c)

```
static void stagger_start(t_philo *philo)
{
    t_data *data;
    long long delay;
    int n;
    int id;

    data = philo->data;
    delay = data->time_to_eat / 2;
    if (delay < 1)
        delay = 1;
    n = data->nb_philos;
    id = philo->id;
    if (n % 2 == 0)
    {
        if (id % 2 == 1)
            ft_usleep(delay, data);
    }
}
```

```

else
{
    if (id == n)
        return ;
    if (id % 2 == 1)
        ft_usleep(delay, data);
}
}

```

El problema que resuelve: todos los hilos arrancan a la vez (gracias a la barrera). Sin desfase, todos intentan coger sus tenedores al mismo tiempo. Con el lock-ordering eso no causa deadlock pero sí causa contención brutal: el filo 1 tiene fork[0], el filo 2 espera fork[1], el filo 3 tiene fork[2]... y los pares no consiguen comer mientras los impares sí, o al revés. Lo arreglamos con un desfase inicial.

Estrategia para N par:

- Pares (id 2, 4, 6...) arrancan inmediatamente.
- Impares (id 1, 3, 5...) duermen `time_to_eat / 2` antes de empezar.

Con N par, los pares cogen los tenedores 1-2, 3-4, 5-6... (que son disjuntos) y comen a la vez. Cuando terminan, los impares cogen los tenedores 0-1, 2-3, 4-5... (también disjuntos). Patrón alternante estable.

Estrategia para N impar:

Con N impar no hay una partición limpia. Aquí se complica. Lo que hacemos:

- El último filósofo (id == N, que es impar cuando N es impar) arranca inmediatamente (sin delay).
- Los demás impares (id 1, 3, 5, ..., N-2) duermen `time_to_eat / 2`.
- Los pares (id 2, 4, ..., N-1) arrancan inmediatamente.

Por qué el último filo arranca sin delay: rompe la simetría circular. Con N=5, normalmente comerían los filos 2 y 4 a la vez (disjuntos), luego 1, 3 y 5 querían entrar, pero 1 y 5 comparten fork[0], y 3 y 5 comparten fork[4]. El filo 5 se quedaría siempre el último. Al arrancarlo con los pares, el filo 5 entra antes que 1 y 3.

Por qué delay = time_to_eat / 2 y no time_to_eat completo: si durmiéramos `time_to_eat` (200ms), gastaríamos demasiado margen del `time_to_die`. Con `time_to_die=610` y `stagger=200`, el filo impar empieza a `t=200` y debe terminar su primera comida antes de `t=610`. Apurado pero posible. Con `stagger=100` (= `time_to_eat/2`) tiene mucho más margen.

9. El ciclo eat → sleep → think

philo_eat (actions.c)

```

int philo_eat(t_philo *philo)
{
    t_data *data;

    data = philo->data;
    if (take_forks(philo))
        return (1);
    pthread_mutex_lock(&data->meal_mutex);
    philo->last_meal_time = get_time_ms();
    philo->meals_eaten++;
    pthread_mutex_unlock(&data->meal_mutex);
    print_status(philo, "is eating");
}

```

```

    ft_usleep(data->time_to_eat, data);
    put_forks(philos);
    return (is_over(data));
}

```

Paso a paso:

1. **take_forks(philos)** — coge los dos tenedores en orden de id ascendente. Si la simulación terminó mientras esperaba, devuelve 1 y aquí salimos retornando 1.
2. **Actualizar last_meal_time y meals_eaten bajo meal_mutex** — ATÓMICAMENTE. Tomamos el lock, hacemos las dos escrituras, soltamos el lock. El monitor que lee estos campos verá una vista coherente: o el meal anterior o el actual, nunca un estado intermedio.
3. **print_status(philos, "is eating")** — DESPUÉS de actualizar last_meal_time. Es importante: el mensaje "is eating" se imprime cuando ya hemos registrado el tiempo de inicio de la comida.
4. **ft_usleep(time_to_eat, data)** — el filo está comiendo durante este tiempo. Si la simulación termina mientras come, ft_usleep detecta is_over y sale antes (de forma que la muerte se reporta a los <10ms).
5. **put_forks(philos)** — suelta ambos tenedores. ANTES de devolver.
6. **return (is_over(data));** — si la simulación terminó durante nuestra comida, salimos del bucle del routine.

Por qué updateamos last_meal_time ANTES de comer y no después:

Si lo actualizáramos al final, la comida "empieza" para el monitor en el momento incorrecto. El subject dice: "if a philosopher has not **started** eating for time_to_die ms ... they die". Es desde que empezó, no desde que terminó. Por tanto el reset del cronómetro del monitor es al INICIO de la comida.

take_forks (actions.c)

```

int take_forks(t_philo *philos)
{
    pthread_mutex_lock(&philos->first_fork->mutex);
    if (is_over(philos->data))
    {
        pthread_mutex_unlock(&philos->first_fork->mutex);
        return (1);
    }
    print_status(philos, "has taken a fork");
    pthread_mutex_lock(&philos->second_fork->mutex);
    if (is_over(philos->data))
    {
        pthread_mutex_unlock(&philos->second_fork->mutex);
        pthread_mutex_unlock(&philos->first_fork->mutex);
        return (1);
    }
    print_status(philos, "has taken a fork");
    return (0);
}

```

Por qué dos chequeos de is_over: entre coger el primer tenedor y coger el segundo puede pasar tiempo (el segundo lo tiene otro filo). En ese hueco la simulación puede terminar. Comprobamos después de cada adquisición.

Si is_over después del primer fork: soltamos el primer fork y salimos.

Si is_over después del segundo fork: soltamos en orden inverso (segundo, luego primero) y salimos.

Por qué primer/segundo y no izquierdo/derecho: porque first_fork es el de MENOR id (asignado así en init_philos). Coger siempre en orden ascendente de id es lo que garantiza no-deadlock.

El mensaje "has taken a fork" se imprime DESPUÉS de coger el lock, no antes. Porque el mensaje refleja el estado real, y solo es real cuando ya tenemos el lock.

put_forks (actions.c)

```
void put_forks(t_philo *philo)
{
    pthread_mutex_unlock(&philo->second_fork->mutex);
    pthread_mutex_unlock(&philo->first_fork->mutex);
}
```

Suelta en orden INVERSO al de toma: primero second_fork, luego first_fork.

Por qué orden inverso: es buena práctica (simétrico con take) y permite a un vecino que esperaba first_fork (que es de menor id que nuestro second_fork) cogerlo lo antes posible. Si soltáramos al revés, el vecino esperando first_fork tendría que esperar a que liberemos los dos.

philo_sleep (actions.c)

```
int philo_sleep(t_philo *philo)
{
    if (is_over(philo->data))
        return (1);
    print_status(philo, "is sleeping");
    ft_usleep(philo->data->time_to_sleep, philo->data);
    return (is_over(philo->data));
}
```

Simple: 1. Chequea is_over al entrar (puede haber muerto alguien mientras liberábamos los tenedores). 2. Imprime "is sleeping". 3. Duerme time_to_sleep (con detección de stop dentro de ft_usleep). 4. Devuelve is_over.

philo_think (actions.c)

```
int philo_think(t_philo *philo, int silent)
{
    long long think;

    if (is_over(philo->data))
        return (1);
    if (!silent)
        print_status(philo, "is thinking");
    think = compute_think_time(philo);
    if (think > 0)
        ft_usleep(think, philo->data);
    return (is_over(philo->data));
}
```

Pensar tiene un detalle clave: el filo NO solo imprime "is thinking" y pasa a intentar comer. Si el sistema está al límite, podría intentar comer DEMASIADO RÁPIDO y robarle el turno a un vecino más hambriento. Por eso calculamos un think_time y dormimos exactamente ese tiempo.

silent: parámetro para suprimir el print de "is thinking". En el mandatory no lo usamos (pasamos 0). Existe por simetría con un posible bonus.

compute_think_time (actions.c)

```
static long long compute_think_time(t_philo *philo)
{
    t_data    *data;
    long long think;
    long long since_last_meal;
    long long max_safe;

    data = philo->data;
    pthread_mutex_lock(&data->meal_mutex);
    since_last_meal = get_time_ms() - philo->last_meal_time;
    pthread_mutex_unlock(&data->meal_mutex);
    max_safe = data->time_to_die - since_last_meal - data->time_to_eat;
    if (max_safe <= 0)
        return (0);
    think = data->time_to_eat - data->time_to_sleep;
    if (think < 0)
        think = 0;
    if (think > max_safe / 2)
        think = max_safe / 2;
    return (think);
}
```

Conceptualmente:

El "think ideal" es $\text{time_to_eat} - \text{time_to_sleep}$. Por qué:

- Un ciclo completo de un filo: eat + sleep + think.
- Para mantener cadencia estable con el vecino con quien compartes fork: eat + sleep + think = 2 * eat (porque el vecino tarda eat en comer mientras tú esperas).
- Despejando: think = 2 * eat - sleep - eat = eat - sleep.

Caso típico: time_to_eat=200, time_to_sleep=200 → think=0. Sin forzar pensamiento. Justo lo que queremos cuando hay margen.

Caso crítico: time_to_eat=200, time_to_sleep=100 → think=100. Forzamos un hueco para que el vecino pueda comer.

max_safe clamp: nunca pensar más de la mitad del tiempo que nos queda hasta morir.

- since_last_meal = cuánto tiempo llevamos sin comer.
- max_safe = time_to_die - since_last_meal - time_to_eat. Es decir: cuánto tiempo nos queda DESPUÉS de pensar para empezar a comer sin morir.
- Si $\text{max_safe} \leq 0$, ya estamos en territorio crítico, no pensamos nada.
- Si $\text{think} > \text{max_safe}/2$, lo recortamos. Así dejamos margen.

Por qué leemos last_meal_time bajo meal_mutex: porque el filo escribe ese campo bajo meal_mutex. Sin el lock, podríamos leer un valor a medio escribir (poco probable en x86 pero teóricamente UB).

10. El monitor (monitor_routine)

monitor_routine (monitor.c)

```
void *monitor_routine(void *arg)
{
    t_data *data;

    data = (t_data *)arg;
    while (!threads_ready(data))
        usleep(50);
    while (1)
    {
        if (is_over(data))
            break;
        if (check_death(data))
            break;
        if (check_all_ate(data))
            break;
        usleep(200);
    }
    return (NULL);
}
```

Estructura:

1. Espera en la barrera (igual que los filos).
2. Bucle infinito polleando cada 200µs:
 - is_over (por si otro hilo ya marcó stop por alguna razón) → sale.
 - check_death (¿murió alguien?) → si sí, ya se marcó stop y se imprimió "died" dentro de la función. Sale.
 - check_all_ate (¿han comido todos suficientes veces?) → si sí, se marca stop. Sale.
3. Retorna. main hace pthread_join sobre nosotros y desbloquea.

Por qué usleep(200): 200µs entre checks. Trade-off:

- Más rápido (50µs): detecta muertes antes pero gasta más CPU haciendo mutex lock/unlock en check_death (uno por filo).
- Más lento (1000µs): puede detectar la muerte hasta 1ms tarde, sumando jitter de hilos, podría no cumplir el "<10ms" del subject.
- 200µs es un buen balance: detección típica <500µs desde la muerte.

check_death (monitor.c)

```
static int check_death(t_data *data)
{
    int i;
    long long elapsed;
    long long last;

    i = 0;
    while (i < data->nb_philos)
    {
        pthread_mutex_lock(&data->meal_mutex);
        last = data->philos[i].last_meal_time;
        pthread_mutex_unlock(&data->meal_mutex);
        elapsed = get_time_ms() - last;
        if (elapsed > data->time_to_die)
```

```

    {
        announce_death(data, data->philos[i].id);
        return (1);
    }
    i++;
}
return (0);
}

```

Para cada filo:

1. Lee last_meal_time bajo meal_mutex (copia local).
2. Suelta el lock inmediatamente. La comparación se hace fuera del lock.
3. Calcula elapsed = now - last.
4. Si elapsed > time_to_die → muerte.

Por qué > estricto y no >=:

El subject dice "if a philosopher has not started eating for time_to_die ms ... they die". En el instante EXACTO $t = \text{time_to_die}$ el filo NO ha estado time_to_die ms sin comer; han pasado time_to_die ms pero la condición es "MÁS DE" time_to_die.

Usar $\geq$ causaría muertes falsas justo en $t = \text{time_to_die}$. Por ejemplo en 5 800 200 200 7, un filo come, luego eat(200) + sleep(200) + ... y si su ciclo siguiente coincide en $t = 800$ EXACTOS, con $\geq$ muere. Con $>$ no.

Por qué leer dentro del lock pero comparar fuera: maximizar el tiempo que el lock está libre. La comparación es trivial; no necesita el lock. Esto reduce contención porque los filos también necesitan el meal_mutex para actualizar.

announce_death (monitor.c)

```

static void announce_death(t_data *data, int philo_id)
{
    pthread_mutex_lock(&data->print_mutex);
    pthread_mutex_lock(&data->stop_mutex);
    if (!data->stop)
    {
        data->stop = 1;
        printf("%lld %d died\n", get_time_ms() - data->start_time, philo_id);
    }
    pthread_mutex_unlock(&data->stop_mutex);
    pthread_mutex_unlock(&data->print_mutex);
}

```

Por qué ANIDA print_mutex y stop_mutex aquí:

Necesitamos garantizar que: 1. Solo se imprima "died" UNA vez aunque el check pase dos veces. 2. Después de imprimir "died", ningún otro mensaje (eating, sleeping, thinking) se imprima.

Si tomáramos solo stop_mutex y luego print_mutex en orden separado, un filo podría intercalar su mensaje "is eating" entre nuestro stop=1 y nuestro printf "died".

Anidando print → stop:

- Tomamos primero print_mutex (bloqueamos todos los prints).
- Luego tomamos stop_mutex (bloqueamos lecturas de stop).
- Si stop ya es 1 (otro thread también pilló la muerte), no imprimimos (evita "died" doble).
- Si stop es 0: lo ponemos a 1 + imprimimos "died" + soltamos ambos.

¿No hay riesgo de deadlock con print_status? No. Lock-ordering: aquí SIEMPRE print → stop. En print_status SIEMPRE solo print (no anida stop). El orden es consistente.

check_all_ate (monitor.c)

```
static int check_all_ate(t_data *data)
{
    int i;
    int count;
    int meals;

    if (data->must_eat == -1)
        return (0);
    count = 0;
    i = 0;
    while (i < data->nb_philos)
    {
        pthread_mutex_lock(&data->meal_mutex);
        meals = data->philos[i].meals_eaten;
        pthread_mutex_unlock(&data->meal_mutex);
        if (meals >= data->must_eat)
            count++;
        i++;
    }
    if (count == data->nb_philos)
    {
        pthread_mutex_lock(&data->stop_mutex);
        data->stop = 1;
        pthread_mutex_unlock(&data->stop_mutex);
        return (1);
    }
    return (0);
}
```

Si must_eat == -1 (no se proporcionó), siempre devuelve 0.

Si se proporcionó: cuenta cuántos filos han comido >= must_eat. Si TODOS han comido suficientes veces → marca stop=1 y devuelve 1.

Por qué NO imprime nada: terminar porque todos han comido es una terminación normal, NO una muerte. El subject no pide ningún mensaje final.

Por qué stop_mutex separado y sin print_mutex: aquí no imprimimos nada, así que no hay que coger print_mutex. Y stop_mutex sólo se toma brevemente para escribir.

11. Utilidades (tiempo, impresión, errores)

get_time_ms (utils.c)

```
long long get_time_ms(void)
{
    struct timeval tv;

    gettimeofday(&tv, NULL);
}
```

```

    return ((long long)tv.tv_sec * 1000 + (long long)tv.tv_usec / 1000);
}

```

gettimeofday rellena tv_sec (segundos desde epoch) y tv_usec (microsegundos extra dentro del segundo, 0..999999).

Convertimos a ms: $\text{sec} * 1000 + \text{usec} / 1000$.

Por qué (long long)tv.tv_sec * 1000 con cast: tv_sec es time_t, en 32-bit suele ser int32_t. int32_t * 1000 puede overflowear si el año >2038. Casteamos a long long antes de multiplicar para evitar UB.

Por qué no clock_gettime con CLOCK_MONOTONIC: sería mejor porque no se ve afectado por cambios del reloj del sistema. Pero el subject recomienda gettimeofday y todos los test scripts asumen su uso.

get_elapsed (utils.c)

```

long long get_elapsed(t_data *data)
{
    return (get_time_ms() - data->start_time);
}

```

Devuelve cuánto tiempo ha pasado desde el inicio de la simulación. Usado en print_status para el timestamp de los mensajes.

ft_usleep (utils.c)

```

void ft_usleep(long long ms, t_data *data)
{
    long long target;
    long long remaining;

    if (ms <= 0)
        return ;
    target = get_time_ms() + ms;
    while (1)
    {
        if (data && is_over(data))
            return ;
        remaining = target - get_time_ms();
        if (remaining <= 0)
            return ;
        if (remaining > 10)
            usleep(2000);
        else if (remaining > 2)
            usleep(500);
        else
            usleep(50);
    }
}

```

Por qué no usleep(ms * 1000) directo:

- usleep no garantiza despertarse en ese momento exacto. Es "al menos" ese tiempo. Con carga alta puede dormir más.
- No detectaría is_over durante el sleep → muerte reportada tarde.

Cómo funciona el sleep adaptativo:

- Si quedan >10ms: dormimos 2000µs (2ms). Suficientemente fino para reaccionar pero no derrocha CPU.
- Si quedan 3-10ms: dormimos 500µs. Acortamos para no sobrepasar.
- Si quedan <3ms: 50µs. Casi busy-wait, para llegar al target con precisión <100µs.

Detección de stop: cada iteración chequea `is_over`. Si es 1, sale inmediatamente, sin esperar a terminar el sleep.

Por qué data &&: durante la fase de inicialización, podríamos llamar a `ft_usleep` con `data=NULL` si hiciéramos un test simple. El check evita un crash.

print_status (utils.c)

```
void print_status(t_philo *philos, const char *status)
{
    t_data *data;
    char buf[80];
    int len;
    ssize_t r;

    data = philos->data;
    if (is_over(data))
        return ;
    pthread_mutex_lock(&data->print_mutex);
    if (!is_over(data))
    {
        len = snprintf(buf, sizeof(buf), "%lld %d %s\n",
                       get_elapsed(data), philos->id, status);
        if (len > 0)
        {
            r = write(1, buf, len);
            (void)r;
        }
    }
    pthread_mutex_unlock(&data->print_mutex);
}
```

Tres chequeos clave:

1. **`is_over(data)` ANTES de coger `print_mutex`:** early-out barato. Si ya terminó, ni siquiera intentamos coger el lock.
2. **`pthread_mutex_lock(&data->print_mutex)`:** sin él los printf de varios hilos se intercalarían.
3. **`is_over(data)` DESPUÉS de coger `print_mutex`:** double-check. Entre el primer chequeo y coger el lock, la simulación pudo terminar. Si ahora vemos que terminó, NO imprimimos.

Por qué `snprintf` + `write` y no `printf`:

- `printf` internamente coge un lock sobre FILE *`stdout`. Con 200 hilos compitiendo, ese lock añade contención adicional.
- `snprintf` formatea a buffer SIN tocar `stdout`.
- `write` es una syscall directa, atómica respecto a otros writes en el mismo fd (bajo POSIX).
- Resultado: una sola syscall, sin lock interno.

Buffer de 80 bytes: `"%lld %d %s\n"` con time hasta 12 dígitos, id hasta 3, status hasta ~20 (longest: "is sleeping"). 80 es holgado.

(void)r;: glibc moderno marca write con warn_unused_result. El (void)r silencia el warning sin desactivar -Werror.

error_exit (utils.c)

```
int error_exit(const char *msg)
{
    ssize_t r;

    r = write(2, msg, ft_strlen_local(msg));
    (void)r;
    r = write(2, "\n", 1);
    (void)r;
    return (1);
}
```

Imprime mensaje en stderr (fd 2) y devuelve 1. El caller hace return (error_exit(...)) que retorna 1.

Por qué write y no fprintf(stderr,...): la Norma de 42 prohíbe funciones que no estén explícitamente permitidas. printf solo se permite para los mensajes del subject (status del filo). Los errores los imprimimos con write que está siempre permitido.

ft_strlen_local (utils.c)

```
size_t ft_strlen_local(const char *s)
{
    size_t i;

    i = 0;
    while (s[i])
        i++;
    return (i);
}
```

Implementación trivial. Se llama _local para no chocar con strlen de <string.h> (que no podemos usar en la parte mandatory porque la Norma restringe librerías).

12. Limpieza y salida

cleanup (init.c)

```
void cleanup(t_data *data)
{
    int i;

    if (data->forks)
    {
        i = 0;
        while (i < data->nb_philos)
            pthread_mutex_destroy(&data->forks[i++].mutex);
        free(data->forks);
        data->forks = NULL;
    }
}
```

```

if (data->philos)
{
    free(data->philos);
    data->philos = NULL;
}
pthread_mutex_destroy(&data->print_mutex);
pthread_mutex_destroy(&data->stop_mutex);
pthread_mutex_destroy(&data->meal_mutex);
}

```

Limpieza idempotente:

- Si `data->forks` es `NULL`, no hace nada (no llegó a inicializarse).
- Si no es `NULL`, destruye los `N` mutexes de tenedores y libera el array.
- Pone `forks = NULL` para que una segunda llamada a `cleanup` no rompa.
- Igual para `philos`.
- Destruye los 3 mutexes globales.

Por qué `data->forks = NULL` después de `free`: evita `double-free`. Si `init_data` falla a mitad de camino, llamamos a `cleanup` desde `init_data` y también desde `main`. La segunda llamada vería `forks=NULL` y no haría nada.

Por qué no chequeamos `NULL` antes de `pthread_mutex_destroy` de los globales: porque están en el struct `t_data` como valores, no punteros. Siempre existen. Lo único es que si `init_mutexes` falló a mitad de camino, hay mutexes inicializados y otros no. `pthread_mutex_destroy` sobre un mutex no inicializado es UB.

(Mejora posible para robustecer: llevar flags de qué mutexes están inicializados. En la práctica `init_mutexes` libera lo que sí inicializó si falla, así que cuando llegamos a `cleanup` desde `main`, todos están inicializados.)

13. Decisiones de diseño y por qué

Lock-ordering global por id

Alternativas consideradas:

- **Pares cogen izquierda primero, impares derecha primero:** funciona pero solo para `N` par. Para `N` impar el último filo cierra el ciclo y necesita su propio caso especial.
- **Try-lock con backoff:** el filo intenta coger su izquierdo, si no puede tras un timeout suelta y reintenta. Funciona pero introduce spinning y es más complejo.
- **Camarero global:** un mutex extra que tienes que coger antes de intentar coger cualquier tenedor. Simple pero serializa el acceso → no escala a `N` grande.

Por qué elegimos lock-ordering: es simple, no añade más locks, y es la solución clásica de teoría OS para el problema.

Barrera de arranque

Sin barrera: los hilos arrancan según los crea `pthread_create`. Con `N=200` eso son cientos de ms de desfase. El primer filo lleva 100ms comiendo cuando el último ni siquiera ha empezado.

Con barrera: todos arrancan a la vez. `start_time` y `last_meal_time` están seteados ANTES de que ningún filo ejecute.

print con write+snprintf

Con N alto (200 fillos), printf introduce contención en el lock interno de stdout. snprintf+write evita ese lock y reduce la latencia de print_status, dejando más CPU para el monitor.

Polling fino del monitor (200µs)

Subject pide detección de muerte en <10ms. Con polling de 1ms y jitter de scheduler, puede llegar a 5-8ms. Con 200µs queda margen sobrado.

Think time dinámico

En compute_think_time, calculamos cuánto pensar basándonos en cuánto tiempo nos queda hasta morir. Esto adapta automáticamente el comportamiento: si hay margen, no pensamos; si estamos al límite, pensamos lo justo.

14. Preguntas frecuentes en defensa

¿Cómo evitas el deadlock?

Lock-ordering global. Cada hilo coge sus dos tenedores siempre en orden ascendente de id (first_fork = menor id, second_fork = mayor id). Esto rompe la dependencia circular del problema clásico: el último hilo (en lugar de coger izquierda → derecha como los demás) coge derecha → izquierda, porque su derecha tiene id menor que su izquierda.

¿Cómo evitas el starvation?

Tres mecanismos:

1. **Stagger inicial** (stagger_start): los hilos no arrancan todos a la vez. Hay un desfase de time_to_eat/2 entre pares e impares.
2. **Think time dinámico** (compute_think_time): si un hilo lleva poco tiempo sin comer, piensa más para dejar comer al vecino.
3. **Lock-ordering**: garantiza que no haya hilos atascados esperando por siempre.

¿Cómo detectas la muerte en <10ms?

- El monitor pollea cada 200µs.
- check_death compara elapsed = now - last_meal_time > time_to_die.
- En cuanto detecta, llama a announce_death que pone stop=1 e imprime "died" bajo print_mutex.
- Latencia típica: <500µs desde la muerte real.

¿Cómo evitas que un hilo imprima un mensaje después de "died"?

announce_death toma print_mutex ANTES de poner stop = 1 y imprime "died" mientras tiene el mutex.

print_status hace DOBLE chequeo de is_over: uno antes de coger el mutex, y otro después. Si "died" se acaba de imprimir, el segundo chequeo lo detecta y no imprime nada.

¿Hay data races?

No. Todos los accesos concurrentes están protegidos:

- stop y threads_ready → stop_mutex.

- `last_meal_time` y `meals_eaten` → `meal_mutex`.
- `stdout` → `print_mutex`.
- Los tenedores → su propio mutex.

¿Hay riesgo de deadlock entre los mutexes globales?

No, gracias a un orden de locking consistente:

- `announce_death`: `print` → `stop`. Único lugar donde se anidan.
- `print_status`: solo `print`.
- `is_over`, `threads_ready`: solo `stop`.
- `philo_eat`: solo `meal` (independiente de `print` y `stop`).
- Monitor: lee `meal`, lee `stop`, escribe `stop`. Ninguno anidado con otro excepto `announce_death`.

El orden globalmente impuesto es: **print** → **stop** → **meal**, pero como ningún sitio toma los 3, solo importa la regla en `announce_death`.

¿Qué hace `last_meal_time = start_time` al inicio?

Inicializamos `last_meal_time` de cada filo al `start_time` global ANTES de liberar la barrera. Sin esto, el monitor leería `last_meal_time = 0` (porque `init_philos` lo deja a 0) y calcularía `elapsed = now`, matando a todos en la primera iteración.

¿Por qué `>` y no `>=` en `check_death`?

El subject dice "if a philosopher has not started eating for `time_to_die` ms ... they die". En el instante exacto `t = time_to_die` NO ha pasado "más" tiempo del límite; ha pasado el límite. La muerte ocurre cuando se SOBREPASA el límite, no cuando se alcanza.

Usar `>=` causaría muertes falsas justo en el borde con timings ajustados (ej. 5 600 200 200).

¿Por qué `philo_eat` actualiza `last_meal_time` al PRINCIPIO?

Porque el subject mide el tiempo desde que un filo EMPEZÓ a comer hasta que vuelve a empezar. Si lo actualizáramos al final, el cronómetro del monitor estaría midiendo desde el FIN de la comida, sumando 200ms extra de margen artificial. Falsea el comportamiento.

¿Funciona con 1 filósofo?

Sí, caso especial en `lone_philo`. El filo coge 1 tenedor, espera `time_to_die + 10`, y suelta. El monitor detecta la muerte y reporta.

¿Funciona con N = 200 (máximo)?

Sí. Es el caso más exigente. Con timings holgados (800 200 200) pasa 100%. Con timings ajustados (610 200 200) hay casos esporádicos de muerte porque el scheduler no siempre da CPU a tiempo, pero el subject no pide N=200 con timings extremos.

¿Por qué no usar `sched_yield` o variables de condición?

- `sched_yield`: solo da una pista al scheduler, no garantiza nada. En la práctica no mejora el comportamiento.
- Condition variables: serían más eficientes que polling pero complican el código y el subject no las menciona. El polling con `usleep(200)` es perfectamente eficiente: ~5000 wakeups/s en el monitor, irrelevante en CPU moderna.

¿Hay memory leaks?

No. Todo lo alocado con malloc (forks, philos) se libera en cleanup. Todos los mutexes inicializados se destruyen.

Probado con valgrind --leak-check=full ./philos 5 800 200 200 5: "All heap blocks were freed -- no leaks are possible".

¿Cumple la Norma de 42?

Sí:

- Funciones ≤ 25 líneas.
 - ≤ 5 funciones por archivo.
 - ≤ 4 parámetros por función.
 - Sin variables globales (solo static dentro de algunas funciones de archivo).
 - Headers protegidos con #ifndef.
 - Sin for, do-while, switch, goto.
 - Llaves al estilo Allman.
 - Comentarios solo en bloques /* */ o //.
-

Apéndice: ejemplo de output esperado

```
$ ./philos 5 800 200 200 3
0 1 has taken a fork
0 1 has taken a fork
0 1 is eating
0 3 has taken a fork
0 3 has taken a fork
0 3 is eating
100 5 has taken a fork
200 1 is sleeping
200 2 has taken a fork
200 2 has taken a fork
200 2 is eating
...
1600 4 is eating
```

Y termina cuando todos han comido 3 veces (sin mensaje final).

Si pones ./philos 4 310 200 100:

```
0 2 has taken a fork
0 2 has taken a fork
0 2 is eating
0 4 has taken a fork
0 4 has taken a fork
0 4 is eating
200 1 has taken a fork
200 2 is sleeping
...
311 1 died
```

Y el programa termina.

Fin del documento. Léelo de cabo a rabo varias veces, intenta explicarle cada función a alguien (o a un patito de goma) y dominarás el proyecto.